

Understanding Placement of SPM design using OpenLane

INTRODUCTION:

The Placement Lab in OpenLane focuses on positioning synthesized standard cells within the defined floorplan core area while respecting timing, congestion, and design rules. Placement is performed after floorplanning and before clock tree synthesis. OpenLane automates both global placement and detailed placement using open-source tools to ensure optimal cell distribution and routability. This stage aims to minimize wirelength, reduce congestion, and meet timing constraints. The placement output serves as a critical foundation for CTS and routing stages in the RTL-to-GDSII flow.

APPLICATION:

The Placement Lab in OpenLane is used to optimize the physical arrangement of standard cells inside the core area of a digital ASIC design. It helps students, researchers, and designers analyse the impact of cell placement on timing closure, congestion, and power efficiency. This lab is widely applied in educational ASIC flows, RISC-V cores, SoC designs, and open-source silicon prototyping using Sky130 PDK. Efficient placement reduces routing complexity and improves overall chip performance

OBJECTIVE:

After completing this lab, you will be able to:

- Understand the role of placement in the RTL-to-GDSII flow
- Perform global and detailed placement using OpenLane
- Analyse placement density, congestion, and cell distribution
- Verify placement legality and standard cell alignment
- Prepare the design for clock tree synthesis (CTS) and routing

PROCEDURE TO CREATE A LAB:

Step 1: Set up OpenLane flow

1.1: Invoke OpenLane in the terminal using the command: `openlane`

```
coe-4@ip-10-100-19-29:~/OpenLaneUser/designs$ openlane
=====
OpenLane started for user: coe-4
Workspace : /home/coe-4/OpenLaneUser
PDK Root  : /labroot/openlane_pdk
=====
OpenLane Container (ff5509f):/openlane$
```

1.2: Open folder- designs: `cd designs`

Step 2: Set up Project Directory

2.1: Create a directory for your lab: `mkdir <project_name>`

2.2: Open lab directory: `cd <project_name>`

2.3: Create directory for project: `mkdir spm`

2.4: Open project directory: `cd spm`

2.5: Inside project directory, create folder: `mkdir src`

2.6: Open folder: `cd src`

Step 3: Create Design file

3.1: Open vi editor for design with command: `vi spm.v`

Note: for RTL code refer the previous document of Demonstration of floorplan using openlane

Step 4: Create .json file

4.1: Open vi editor: `vi config.json`

```
{
  "DESIGN_NAME": "spm",
  "VERILOG_FILES": "dir::src/*.v",
  "CLOCK_PERIOD": 10,
  "CLOCK_PORT": "clk",
  "CLOCK_NET": "ref::$CLOCK_PORT",
  "FP_PDN_VOFFSET": 7,
  "FP_PDN_HOFFSET": 7,
  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
  "FP_PDN_SKIPTRIM": true,
  "pdk::sky130*": {
    "FP_CORE_UTIL": 45,
```

```
"scl::sky130_fd_sc_hd": {
    "CLOCK_PERIOD": 10
},
"scl::sky130_fd_sc_hd11": {
    "CLOCK_PERIOD": 10
},
"scl::sky130_fd_sc_hs": {
    "CLOCK_PERIOD": 8
},
"scl::sky130_fd_sc_ls": {
    "CLOCK_PERIOD": 10,
    "MAX_FANOUT_CONSTRAINT": 5
},
"scl::sky130_fd_sc_ms": {
    "CLOCK_PERIOD": 10
}
},
"pdk::gf180mcu*": {
    "CLOCK_PERIOD": 24.0,
    "FP_CORE_UTIL": 40,
    "MAX_FANOUT_CONSTRAINT": 4,
    "PL_TARGET_DENSITY": 0.5
}
}
```

Step 5: Use the following commands to run the flow:

OpenLane to start the design flow in interactive mode

```
i) flow.tcl -interactive
```

Used in the OpenLane interactive shell to prepare your design for the digital IC design flow.

```
ii) prep -design spm
```

Runs yosys synthesis on the current design.

```
iii) run_synthesis
# To initiate Floorplanning
iv) run_floorplan
# To initiate Placement
V) run_placement
```

```
% run_placement
[STEP 7]
[INFO]: Running Global Placement (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/placement/7-global.log)...
[STEP 8]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/placement/8-gpl_sta.log)...
[STEP 9]
[INFO]: Running Placement Resizer Design Optimizations (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/placement/9-resizer.log)...
[STEP 10]
[INFO]: Running Detailed Placement (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/placement/10-detailed.log)...
[STEP 11]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/placement/11-dpl_sta.log)...
%
```

Step 6: View placement using klayout:

Locate the DEF file

```
i) designs/spm/runs/RUN_2026.01.23_05.16.33/results/placement/spm.def
ii) designs/spm/runs/RUN_2026.01.23_05.16.33/tmp/merged.nom.lef
```

Open in klayout in the same directory where spm.def is located

```
ii) klayout spm.def merged.nom.lef
```

NOTE: keep both file spm.def and merged.nom.lef in the same directory , for that you have to copy spm.def from result directory to tmp directory created inside runs directory.

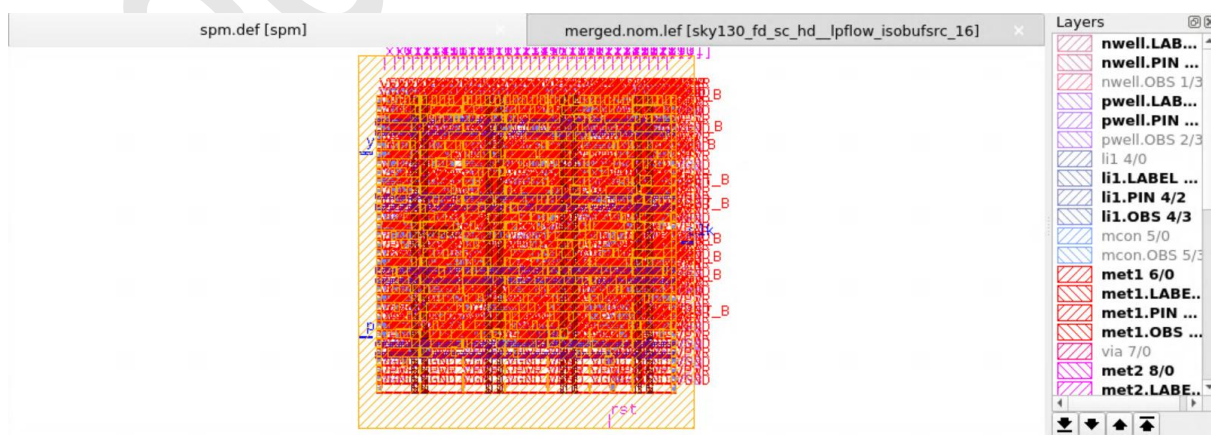


Figure 1 : klayout view of placement.def

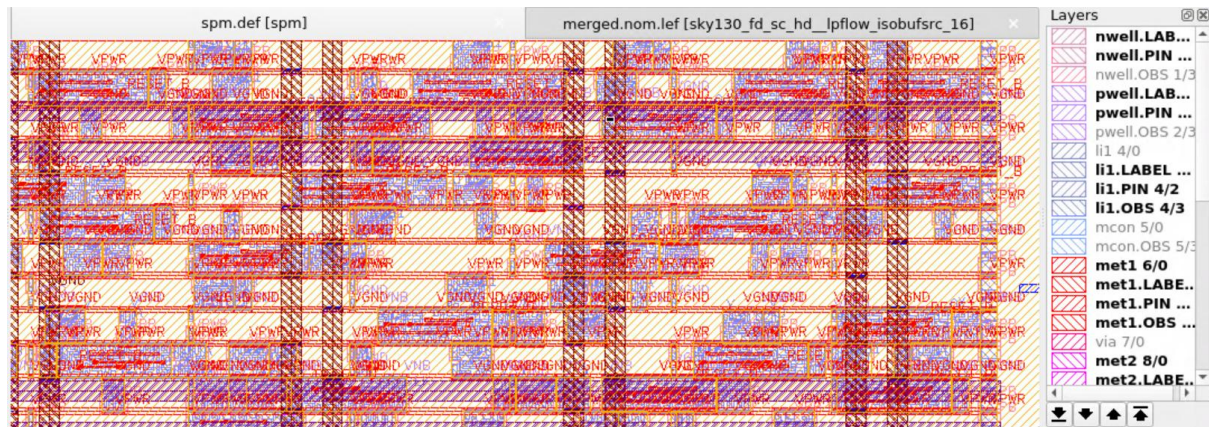


Figure 2 : Zoom in klayout view of placement.def

Note :Zoomed view of placement.def when metal layer are off.

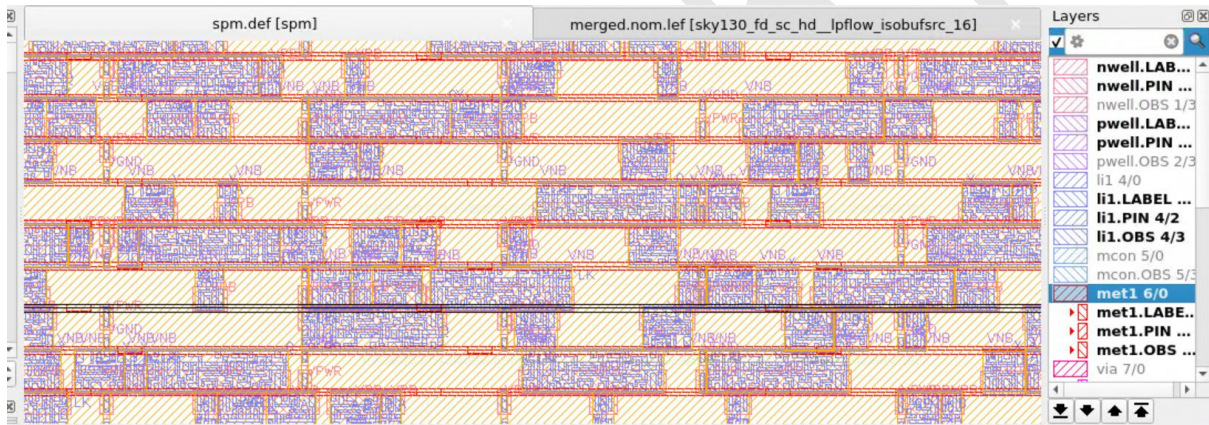


Figure 3: Zoom in layout view of placement.def

OBSERVATION:

- Distribution of standard cells across the core
- Placement density and utilization
- Absence of overlapping or illegal cell placements
- Alignment of standard cells with rows
- Congestion visualization in dense regions
- Placement DEF visualization in klayout

OUTPUT:

Output of the floorplan stage is a placement.def that shows:

- Placed standard cells within the core area
- Optimized cell distribution

- Legalized and row-aligned placement
- Reduced wirelength and congestion
- Placement ready for CTS

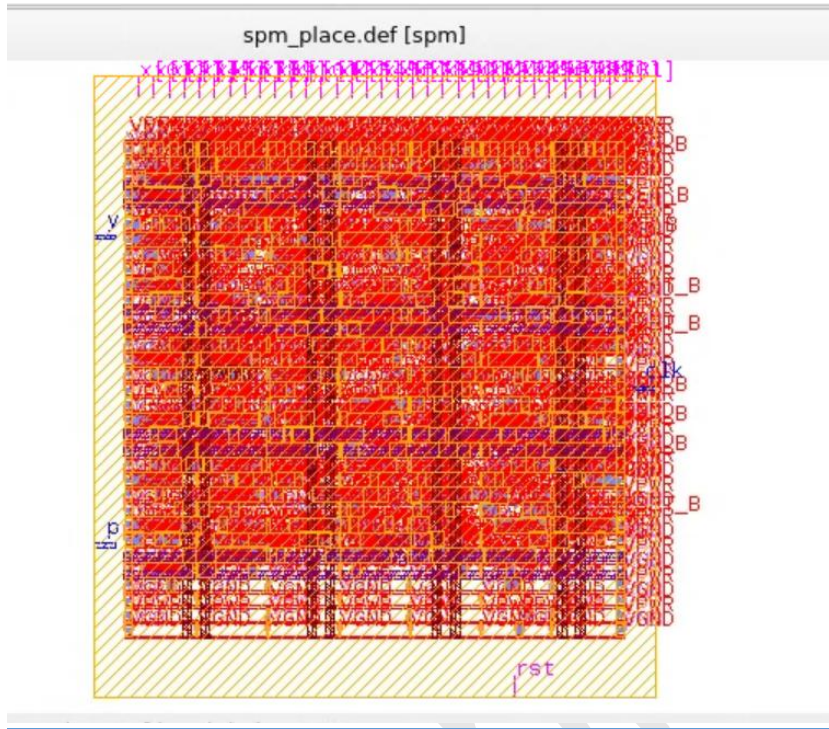


Figure 4: Output of placement.def

CONCLUSION:

In this placement lab, OpenLane was successfully used to perform global and detailed placement of standard cells for the given RTL design. The placement stage effectively distributed cells within the predefined floorplan, ensured legal alignment with standard cell rows, and minimized congestion and wirelength. The placement DEF was verified using KLayout, confirming proper cell distribution and design rule compliance. This successful placement provides a robust foundation for subsequent clock tree synthesis and routing stages, enabling efficient timing closure and manufacturable ASIC design.